

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Теория алгоритмов
ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ
РАБОТЫ №1

«Реализация двумерного клеточного автомата»

Вариант 32

Студент,
группы 5130201/30002

_____ Михайлова А. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2025 г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
1.1 Клеточный автомат	4
1.2 Классификация по Вольфраму	6
2 Особенности реализации	8
3 Результаты работы программы	13
4 Анализ двумерного клеточного автомата	15
Заключение	18
Список использованной литературы	19

Введение

В лабораторной работе необходимо было реализовать двумерный клеточный автомат с окрестностью фон Неймана. Граничные условия – торроидальные. Пользователь определяет ширину поля и количество итераций. Необходимо учесть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя. В результате необходимо было проанализировать свой клеточный автомат в отчете (его поведение, паттерны, "сходимость" и т.д.).

Полученный номер правила для 32 варианта: 152444160_{10}

1 Математическое описание

1.1 Клеточный автомат

Клеточные автоматы – это регулярная структура динамических объектов (клеток), функционирующих синхронно. Клеточные автоматы моделируют процессы, разворачивающиеся в дискретном пространстве и в дискретном времени.

Клеточный автомат имеет состояние, определяемое как набор (вектор или матрица) состояний компонентных автоматов. Каждый автомат – это автомат без выхода, выходом является его состояние. Вход на каждом шаге клеточного автомата – состояния его соседей. На следующем шаге (в следующем такте) новое состояние каждого автомата определяется как функция его текущего состояния и текущих состояний его соседей.

Двумерный клеточный автомат можно определить как множество конечных автоматов на плоскости, помеченных целочисленными координатами (i, j) , каждый из которых может находиться в одном из состояний.

Двумерный клеточный автомат формально задается кортежем $A = (L, S, N, f)$, где L – двумерная решётка клеток с целочисленными координатами (i, j) , $S = \{s_1, s_2, \dots, s_k\}$ – конечное множество возможных состояний клетки (в данной работе $S = \{0, 1\}$), N – функция, определяющая окрестность клетки (в работе используется окрестность фон Неймана). Окрестность фон Неймана для клетки (i, j) на решетке можно представить как множество соседних клеток, имеющих общую сторону, что описывается формулой: $N(i, j) = \{(i - 1, j), (i, j - 1), (i, j + 1), (i + 1, j)\}$. f – правило перехода конечного автомата ($f = 1001000101100001110100000000_2$).

Основой клеточного автомата является пространство из прилегающих друг к другу клеток, образующих решётку. Каждая клетка C_i может находиться в одном из конечного множества состояний $S = s_1, s_2, \dots, s_k$. Состояние каждой клетки определяется заданными правилами перехода. Правила рассчитывают следующее состояние клетки из текущего состояния клетки и состояния соседних клеток.

Состояние клетки с координатами (i, j) определено для всех целых i, j , но считается эквивалентным состоянию клетки с нормализованными координатами: $(i \bmod v, j \bmod h)$, где операция $\bmod$ понимается в неотрицательной арифметике по модулю, v – число строк, h – число столбцов.

Для центральной клетки $c_{i,j}$ её пять состояний на текущем шаге (сама

центральная клетка и четыре ближайших соседа) вычисляются как:

$$\begin{aligned}s_0 &= x_{i,j} \\ s_1 &= x_{i,(j-1) \bmod h} \\ s_2 &= x_{(i-1) \bmod v,j} \\ s_3 &= x_{i,(j+1) \bmod h} \\ s_4 &= x_{(i+1) \bmod v,j}\end{aligned}$$

где $x_{i,j} \in S = \{0, 1\}$ – состояние клетки в i -й строке и j -м столбце, причём индексы строк i и столбцов j принимают значения из множеств $Z_v = \{0, 1, \dots, v - 1\}$ и $Z_h = \{0, 1, \dots, h - 1\}$ соответственно.

Окрестность фон Неймана клетки – совокупность четырёх клеток на квадратном паркете, имеющих общую сторону с данной клеткой. На рисунке 1 представлено обозначение клеток в данной лабораторной работе.

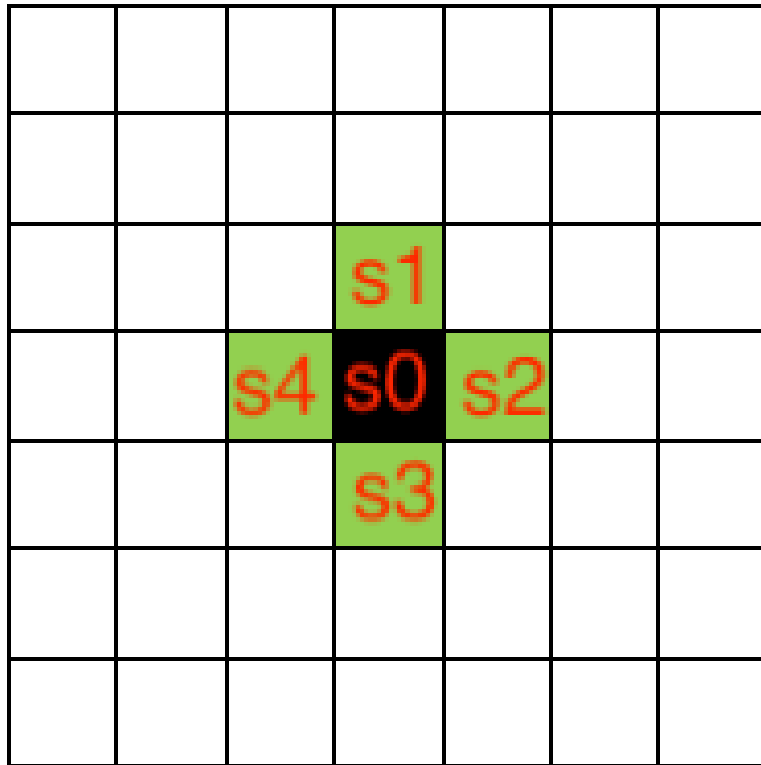


Рис. 1: Окрестность фон Неймана

Один шаг автомата подразумевает обход всех клеток и на основе данных о текущем состоянии клетки и её окрестности определение нового состояния клетки, которое будет у неё при следующем шаге. Перед стартом автомата оговаривается начальное состояние клеток, которое может устанавливаться целенаправленно или случайным образом. В данной работе функция представлена числом $11 \cdot 27 \cdot 8 \cdot 2005 \cdot 32 = 152444160_{10} = 1001000101100001110100000000_2$. Так как нужна функция 5 переменных, то двоичная форма числа дополняется нулями в старших разрядах. Все переходы состояний представлены на рисунке 2.

	s0	s1	s2	s3	s4	f
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	1	0	0
3	0	0	0	1	1	0
4	0	0	1	0	0	1
5	0	0	1	0	1	0
6	0	0	1	1	0	0
7	0	0	1	1	1	1
8	0	1	0	0	0	0
9	0	1	0	0	1	0
10	0	1	0	1	0	0
11	0	1	0	1	1	1
12	0	1	1	0	0	0
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	0
16	1	0	0	0	0	0
17	1	0	0	0	1	0
18	1	0	0	1	0	0
19	1	0	0	1	1	1
20	1	0	1	0	0	1
21	1	0	1	0	1	1
22	1	0	1	1	0	0
23	1	0	1	1	1	1
24	1	1	0	0	0	0
25	1	1	0	0	1	0
26	1	1	0	1	0	0
27	1	1	0	1	1	0
28	1	1	1	0	0	0
29	1	1	1	0	1	0
30	1	1	1	1	0	0
31	1	1	1	1	1	0

Рис. 2: Таблица переходов состояний

1.2 Классификация по Вольфраму

Стивен Вольфрам в своей книге *A New Kind of Science* предложил 4 класса, на которые все клеточные автоматы могут быть разделены в зависимости от типа их эволюции. Классификация Вольфрама была первой попыткой классифицировать сами правила, а не типы поведения правил по отдельности. В порядке возрастания сложности классы выглядят следующим образом:

Класс 1: Результатом эволюции начальных условий является быстрый переход к гомогенной стабильности. Любые негомогенные конструкции быстро исчезают.

Класс 2: Результатом эволюции начальных условий является быстрый переход в неизменяемое негомогенное состояние либо возникновение циклической последовательности. Большинство структур начальных условий быстро исчезает, но некоторые остаются. Локальные изменения в начальных условиях оказывают локальный характер на дальнейший ход эволюции системы.

Класс 3: Результатом эволюции почти всех начальных условий являются псевдо-случайные, хаотические последовательности. Любые стабильные структуры, которые возникают почти сразу же уничтожаются окружающим их шумом. Локальные изменения в начальных условиях оказывают неопределяемое влияние на ход эволюции системы.

Класс 4: Результатом эволюции являются структуры, которые взаимодействуют сложным образом с формированием локальных, устойчивых структур. В результате эволюции могут получаться некоторые последовательности Класса 2, описанного выше. Локальные изменения в начальных условиях оказывают неопределяемое влияние на ход эволюции системы.

2 Особенности реализации

Клеточный автомат в программе задан строкой *const string RULE_STRING* = "00001001000101100001110100000000".

Функция getNextState вычисляет новое состояние центральной клетки по заданному правилу клеточного автомата.

Вход: клетки s0, s1, s2, s3, s4

Выход: новое состояние центральной клетки

```
1 int getNextState(int s0, int s1, int s2, int s3, int s4
  ) {
2     int index = (s0 << 4) | (s1 << 3) | (s2 << 2) |
      (s3 << 1) | s4;
3     return RULE_STRING[index] - '0';
4 }
```

Функция main – основная функция для работы с клеточным автоматом. Выступает в роли меню для всей программы. В ней происходят проверки на неверный ввод, создание автомата и использование реализованных методов.

Вход: инициализация параметров программы

Выход: результаты работы методов

```
1 int main() {
2     int width, height, iterations;
3     char choice;
4     do {
5         cout << "Введите_ширину_поля_от(1_до_80):_";
6         cin >> width;
7         if (cin.fail() || width < 1 || width > 80) {
8             cout << "Ошибка:_ширина_должна_быть_целым_числом_от_
              1_до_80.\n";
9             cin.clear();
10            cin.ignore(10000, '\n');
11            width = -1;
12        }
13    } while (width < 1 || width > 80);
14    do {
15        cout << "Введите_высоту_поля_от(1_до_80):_";
16        cin >> height;
17        if (cin.fail() || height < 1 || height > 80) {
18            cout << "Ошибка:_высота_должна_быть_целым_числом_от_
              1_до_80.\n";
19            cin.clear();
20            cin.ignore(10000, '\n');
21            height = -1;
```



```

22 }
23 } while (height < 1 || height > 80);
24
25 CellularAutomaton ca(width, height);
26 do {
27     cout << "Выберите начальные условия:\n";
28     cout << "m - вручную\n";
29     cout << "r - случайно\n";
30     cin >> choice;
31
32     if (cin.fail()) {
33         cin.clear();
34         cin.ignore(10000, '\n');
35         choice = '\0';
36     }
37 } while (choice != 'm' && choice != 'M' && choice != 'r'
38         && choice != 'R');
39 if (choice == 'm' || choice == 'M') {
40     ca.setManualInitialState();
41 } else {
42     ca.setRandomInitialState();
43 }
44 do {
45     cout << "Введите количество итераций от (1 и больше): ";
46     cin >> iterations;
47     if (cin.fail() || iterations < 1) {
48         cout << "Ошибка: количество итераций должно быть
49             целым числом >= 1.\n";
50         cin.clear();
51         cin.ignore(10000, '\n');
52         iterations = -1;
53     }
54 } while (iterations < 1);
55 ca.run(iterations);
56 return 0;
57 }

```

Класс CellularAutomaton реализует двумерный клеточный автомат. currentGrid хранит текущее состояние автомата, nextGrid временно хранит новые состояния всех клеток, рассчитанные на основе текущего состояния, width, height хранят размеры поля.

```

1 class CellularAutomaton {
2 private:

```

```

3  int width, height;
4  vector<vector<int>> currentGrid;
5  vector<vector<int>> nextGrid;
6
7  public:
8  CellularAutomaton(int w, int h) : width(w), height(h) {
9  currentGrid.resize(height, vector<int>(width, 0));
10 nextGrid.resize(height, vector<int>(width, 0));
11 }
12 ...};

```

Функция toroidal приводит координату x к корректному индексу в диапазоне [0, size - 1] с учётом тороидальных граничных условий.

Вход: координата, размер

Выход: новый индекс

```

1  int toroidal(int x, int size) {
2      return ((x % size) + size) % size;
3  }

```

Функция getNeighbor возвращает значение соседней клетки с учетом тороидальных граничных условий.

Вход: строка, столбец, смещение по строке, смещение по столбцу

Выход: значение соседней клетки

```

1  int getNeighbor(int row, int col, int dr, int dc) {
2      int nr = toroidal(row + dr, height);
3      int nc = toroidal(col + dc, width);
4      return currentGrid[nr][nc];
5  }

```

Функция setManualInitialState запрашивает у пользователя начальное состояние поля построчно.

Вход: запрос на ручной ввод состояния поля

Выход: начальное состояние поля

```

1  void setManualInitialState() {
2  cout << "Введите состояние каждой клетки в строке через
   пробел (0 или 1): \n";
3  for (int i = 0; i < height; ++i) {
4      cout << "Строка " << i + 1 << ": ";
5      for (int j = 0; j < width; ++j) {
6          cin >> currentGrid[i][j];
7          if (currentGrid[i][j] != 0 &&
   currentGrid[i][j] != 1) {
8              cout << "Ошибка: введите только 0
   или 1. \n";

```

```

9         j--;
10    }
11 }
12 }
13 }

```

Функция `setRandomInitialState` заполняет поле случайными значениями — каждая клетка независимо принимает состояние 0 или 1 с равной вероятностью.

Вход: запрос на случайное заполнение начального состояния поля

Выход: начальное состояние поля

```

1 void setRandomInitialState() {
2     random_device rd;
3     mt19937 gen(rd());
4     uniform_int_distribution<> dis(0, 1);
5
6     for (int i = 0; i < height; ++i) {
7         for (int j = 0; j < width; ++j) {
8             currentGrid[i][j] = dis(gen);
9         }
10    }
11 }

```

Функция `update` выполняет один шаг эволюции клеточного автомата: для каждой клетки она определяет её новое состояние на основе текущего состояния и состояний её четырёх соседей, используя заданное правило, и обновляет всё поле целиком.

Вход: запрос на обновление поля

Выход: обновленное состояние поля

```

1 void update() {
2     for (int i = 0; i < height; ++i) {
3         for (int j = 0; j < width; ++j) {
4             int s0 = currentGrid[i][j];
5             int s1 = getNeighbor(i, j, -1, 0); //
        верх
6             int s2 = getNeighbor(i, j, 0, 1); //
        право
7             int s3 = getNeighbor(i, j, 1, 0); //
        низ
8             int s4 = getNeighbor(i, j, 0, -1); //
        лево
9

```

```

10         nextGrid[i][j] = getNextState(s0, s1,
11         s2, s3, s4);
12     }
13 }
14 currentGrid = nextGrid;
15 }

```

Функция print выводит текущее состояние поля в консоль – построчно, в виде матрицы.

Вход: запрос на вывод поля

Выход: состояние поля, выведенное в консоль

```

1 void print() const {
2     for (int i = 0; i < height; ++i) {
3         for (int j = 0; j < width; ++j) {
4             cout << currentGrid[i][j] << "
5         ";
6         }
7         cout << endl;
8     }
9     cout << "-----" << endl;
10 }

```

Функция run запускает эволюцию клеточного автомата на заданное число итераций.

Вход: число итераций

Выход: шаги эволюции клеточного автомата

```

1 void run(int iterations) {
2     cout << "\Начальное состояние:\n";
3     print();
4
5     for (int iter = 1; iter <= iterations; ++iter)
6     {
7         cout << "Итерация" << iter << ":\n";
8         update();
9         print();
10    }
11 }

```

3 Результаты работы программы

При запуске программы предлагается ввести ширину и высоту поля, а также выбрать как ввести начальные условия – вручную или случайным образом (рис. 3).

```
Введите ширину поля (от 1 до 80): 4
Введите высоту поля (от 1 до 80): 4
Выберите начальные условия:
m – вручную
r – случайно
|
```

Рис. 3: Запуск программы

При выборе m начальные условия вводятся вручную – рис. 4.

```
m
Введите состояние каждой клетки в строке через пробел (0 или 1):
Строка 1: 0 1 0 1
Строка 2: 0 0 0 0
Строка 3: 1 1 1 1
Строка 4: 1 0 0 1
Введите количество итераций (от 1 и больше): |
```

Рис. 4: Ручной ввод

При выборе r - случайно – рис 5.

```
r
Введите количество итераций (от 1 и больше): |
```

Рис. 5: Автоматический ввод

Далее пользователю нужно ввести количество итераций. После ввода выводится состояние клеточного автомата на каждой итерации и программа завершается – рис. 6.

```

Введите количество итераций (от 1 и больше): 3

Начальное состояние:
0 0 0 1
1 0 0 1
1 0 0 0
0 0 0 1
-----
Итерация 1:
0 0 1 0
1 0 1 0
0 0 0 1
0 0 1 0
-----
Итерация 2:
0 1 0 0
0 0 0 1
0 0 1 0
0 1 0 0
-----
Итерация 3:
1 0 0 0
0 0 0 0
0 0 0 0
1 0 0 0
-----
Program ended with exit code: 0

```

Рис. 6: Вывод состояний клеточного автомата

Примеры обработки некорректного ввода представлены на рис. 7, 8.

```

Введите ширину поля (от 1 до 30): e
Ошибка: ширина должна быть целым числом от 1 до 30.

```

Рис. 7: Некорректный ввод

```

Выберите начальные условия:
m – вручную
r – случайно
2
Выберите начальные условия:
m – вручную
r – случайно
|

```

Рис. 8: Некорректный ввод

4 Анализ двумерного клеточного автомата

Клеточный автомат, реализованный в лабораторной, принадлежит к 2 классу, в соответствии с классификацией Вольфрама. Результатом эволюции начальных условий является быстрый переход в неизменяемое негомогенное состояние с циклическими последовательностями. Вывод был сделан исходя из анализа поведения автомата в различных начальных условиях.

При ручном задании автомата 10x10 с 1 живой клеткой, происходит последовательное перемещение живой клетки влево – рис. 9.

Начальное состояние:	Итерация 1:	Итерация 2:	Итерация 3:
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0 0	0 0 0 0 1 0 0 0 0 0 0	0 0 0 0 0 1 0 0 0 0 0	0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0

Рис. 9: Автомат с 1 живой клеткой

Проведены тесты – по 10 тестов для поля 100 на 100 и 20 итераций и для поля 500 на 500 и 100 итераций, которые показали, что при случайном заполнении количество живых клеток всегда падает в первые несколько итераций до 14% от общего количества клеток, и остается стабильным, часто одинаковым на протяжении следующих итераций – рис. 10, 11.

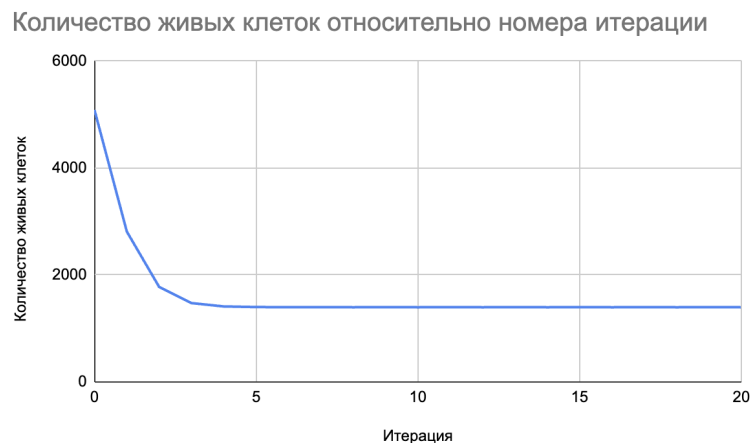


Рис. 10: Автомат 100 на 100

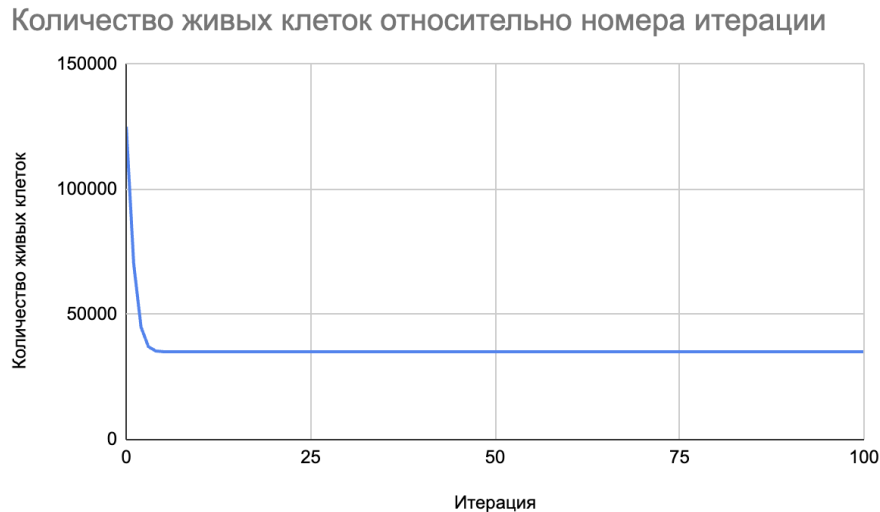


Рис. 11: Автомат 500 на 500

При задании начального состояния автомата в форме буквы X, количество живых клеток быстро снизилось, автомат пришел к стабильному, циклическому состоянию – 2 живых клетки, перемещающихся влево – рис. 12.

Начальное состояние:	Итерация 1:	Итерация 2:	Итерация 3:
1 0 0 0 1	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0
0 1 0 1 0	0 0 1 0 0	0 0 0 0 0	0 0 0 0 0
0 0 1 0 0	0 1 0 1 0	1 0 0 0 0	0 0 0 0 1
0 1 0 1 0	0 0 1 0 0	0 0 0 0 0	0 0 0 0 0
1 0 0 0 1	1 0 0 0 0	0 0 0 0 1	0 0 0 1 0

Рис. 12: Автомат в форме буквы X

При задании начального состояния автомата форме буквы Г произошла та же ситуация – рис 13.

Начальное состояние:	Итерация 1:	Итерация 2:	Итерация 3:
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0
0 1 1 1 0	1 0 1 0 0	0 0 0 0 1	0 0 0 1 0
0 1 0 0 0	1 0 0 0 0	0 0 0 0 1	0 0 0 1 0
0 1 0 0 0	1 0 0 0 0	0 0 0 0 1	0 0 0 1 0
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0

Рис. 13: Автомат в форме буквы Г

Начальное состояние:	Итерация 1:	Итерация 2:	Итерация 3:
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0
1 1 1 1 0	1 1 1 0 0	1 1 0 0 1	1 0 0 1 1
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0
0 0 0 0 0	0 0 0 0 0	0 0 0 0 0	0 0 0 0 0

Рис. 14: Автомат в форме линии 1*4

Заключение

В ходе выполнения лабораторной работы был реализован двумерный клеточный автомат с окрестностью фон Неймана, торроидальными граничными условиями. Пользователь определяет ширину поля и количество итераций. Учтены возможности ввода различных начальных условий: вручную и случайным образом по выбору пользователя. Проанализирован клеточный автомат. Работа выполнена в среде программирования Xcode на языке программирования C++. Использовано онлайн-приложение Google Таблицы для анализа поведения автомата. Поведение клеточного автомата можно причислить к второму классу по классификации Вольфрама. Таблица состояний составлена в соответствии с функцией, представленной числом $f = 152444160_{10}$

Достоинства программы:

- Адаптация программы под любую функцию пяти переменных, путём изменения значения `RULE_STRING`;
- Вычисление нового состояния с использованием битовых операций.

Недостатки программы:

- Ограничен размер поля. Максимальный размер поля клеточного автомата – 80×80 .
- При выборе ручного ввода начального состояния клеточного автомата требуется ввести через пробел и построчно состояние каждой клетки, что может занимать много времени, особенно при больших размерах поля.

Масштабированием программы может стать добавление графического интерфейса (вывод графика зависимости количества клеток от номера итерации в виде диаграммы, ввод состояния автомата на выведенное клеточное поле, а не построчно), возможность задания правила пользователем (пользователь не меняет правило в коде, а может задать его в консоли), подсчет зависимости живых клеток от номера итерации.

Список литературы

- [1] Секция "Телематика"/ текст : электронный / — URL: <https://tema.spbstu.ru/algorithm/> (Дата обращения 05.11.2025).
- [2] «New kind of science», S. Wolfram / текст : электронный / — URL: <https://www.wolframscience.com/> (Дата обращения 05.11.2025).